CSE 344 – System Programming

Homework #3

Multi-Process Word Transportation and Concurrent Sorting System

Student Name: FATİH EMRE OĞAN
Student Number: 230104004090
Due Date: 15 / 04 / 2026

Table of Contents

1. Introduction

1.1 Brief Description of the Problem

This homework implements a multi-process system that reads words from a text file, distributes them across building floors, transports their individual characters between floors via elevators, and reconstructs the original words using concurrent sorting processes.

1.2 Overview of the System

The system simulates a building with multiple floors. Each word has a unique ID, an arrival floor (determined by which word-carrier process claims it), and a designated sorting floor defined in the input file. Words are decomposed into characters; each character is transported independently and placed into a sorting area, where sorting processes reconstruct the original word.

The system consists of six process types:

- Parent / Coordinator Process
- Word-Carrier Processes (per floor)
- Letter-Carrier Processes (mobile)
- Sorting Processes (per floor)
- Delivery Elevator Process
- Reposition Elevator Process

1.3 Main Challenges

The primary challenges in this assignment are:

- Concurrency: Multiple processes operate simultaneously on shared data structures without causing race conditions.
- Synchronization: POSIX semaphores must guard all shared state – word flags, elevator queues, floor capacities, and sorting areas.
- Process Coordination: The parent must detect when all words are complete and orchestrate a clean shutdown of all child processes.
- Deadlock Avoidance: A consistent lock acquisition order must be maintained across all process types.

2. System Architecture

2.1 Process Types and Responsibilities

2.1.1 Parent / Coordinator Process

The parent process is responsible for reading the input file, initializing shared memory and all synchronization primitives, spawning all child processes (word-carriers, letter-carriers, sorting processes, and both elevators), monitoring system completion, terminating all processes on completion or SIGINT, generating the final output file, and printing summary statistics.

All child processes in the system are created exclusively by the parent. No child process creates additional system-level processes.

2.1.2 Word-Carrier Processes

Word-carrier processes scan the input word list in synchronized round-robin order. Each process is permanently assigned to a single floor (its arrival floor). When a word is selected, the process attempts all-or-nothing admission: both the arrival floor and the sorting floor must have available capacity. If admission succeeds, the word's admitted flag is set and letter-carriers on the arrival floor are notified. If admission fails, the claimed flag is cleared and the word is released for future attempts.

2.1.3 Letter-Carrier Processes

Letter-carrier processes are mobile: they start on an assigned floor but may move to other floors during execution. Each process scans for unclaimed character tasks on its current floor. If a task is found and the destination is different from the current floor, the letter-carrier requests the delivery elevator, waits for arrival, then places the character in the word's sorting area. If the destination is the same floor, the character is placed directly. After delivery, the letter-carrier remains on the destination floor and searches for new work. If no work is available, it requests the reposition elevator to move to a random floor.

If any floor has no letter-carrier processes at any time, the parent process immediately spawns the configured number of new letter-carrier processes on that floor.

2.1.4 Sorting Processes

Sorting processes are permanently bound to their initial sorting floor. Multiple sorting processes may exist on the same floor, but only one may operate on a specific word at a time (enforced via per-word semaphore). Each sorting process scans admitted, incomplete words on its floor, acquires the per-word lock with a non-blocking try, and performs a sorting pass: characters already in their correct positions are fixed; misplaced characters are moved to empty target slots or swapped with non-fixed characters. When all positions are fixed, the word is marked complete.

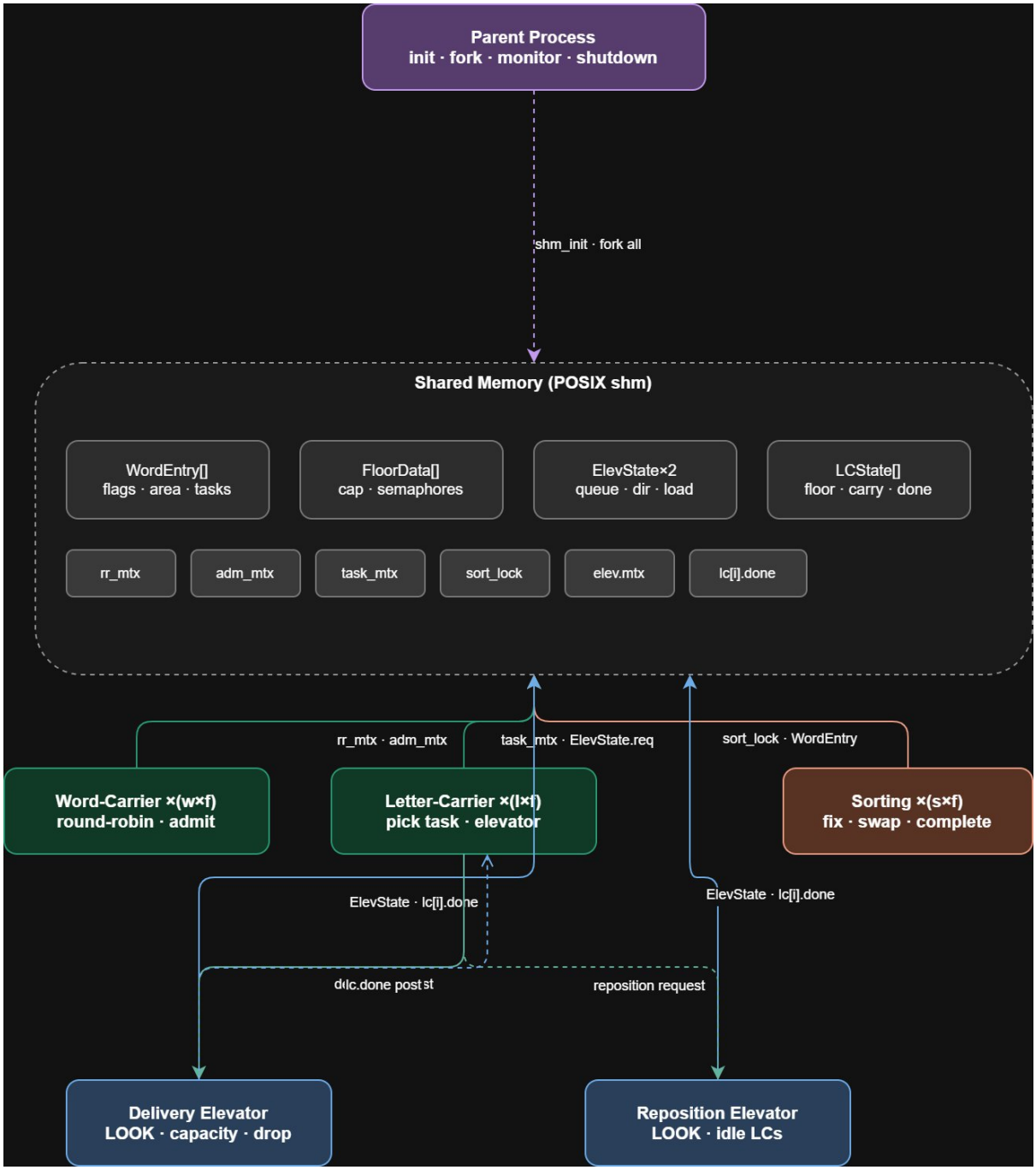
2.1.5 Delivery Elevator Process

The delivery elevator handles transportation of letter-carriers carrying characters. It uses the LOOK algorithm: it moves in one direction until there are no more passengers to drop off and no pending requests in that direction, then reverses. When idle with no requests, it waits on a semaphore. The elevator enforces a configurable capacity limit on simultaneous passengers.

2.1.6 Reposition Elevator Process

The reposition elevator operates identically to the delivery elevator but handles only idle letter-carriers (those with no character to transport). It uses a separate request queue and moves carriers to their requested random destination floors.

2.2 Process Interaction Diagram



2.3 Process Communication

All inter-process communication is performed through POSIX shared memory. Processes do not use pipes, sockets, or message queues. Synchronization is achieved exclusively through POSIX semaphores embedded within the shared memory segment.

3. Data Structures and Shared Memory Design

3.1 Shared Memory Organization

The entire system state is stored in a single POSIX shared memory segment named after the parent PID (e.g., /hw3_shm_<pid>). The segment contains: the global configuration, all word entries, per-floor data, elevator states, letter-carrier states, global semaphores, and statistics.

3.2 Word Structure (WordEntry)

Each word is represented by a WordEntry structure containing:

- orig[]: the original word string (used as the reconstruction reference)
- area[]: the sorting area – characters are placed here as they arrive
- occ[]: occupied flags – 1 if the slot has a character
- fix[]: fixed flags – 1 if the character at this slot is permanently in its correct position
- claimed: set to 1 when a word-carrier selects the word
- admitted: set to 1 when the word is successfully admitted to both floors
- completed: set to 1 when all fix[] entries are 1
- tasks[]: one CharTask per character, each storing the character, its original index, destination floor, claimed/delivered flags, and carrier ID
- sort_lock: per-word semaphore ensuring only one sorter operates on this word at a time

3.3 Elevator State (ElevState)

Each elevator has an ElevState structure containing: current floor, direction (1=UP, -1=DOWN, 0=idle), capacity, current passengers (load_lc[] and load_to[] arrays), a sparse request queue (req[]), a mutex semaphore (mtx) protecting all fields, and an availability semaphore (avail) posted when a new request arrives.

3.4 Per-Floor Data (FloorData)

Each floor has: word_count (number of active words, guarded by adm_mtx), task_mtx (protects character-task scanning and claiming), lc_work (posted when new character tasks arrive), sort_work (posted when a character is placed in a sorting area), lc_cnt (number of letter-carriers currently on this floor), and lc_mtx (protects lc_cnt).

3.5 Letter-Carrier State (LCState)

Each letter-carrier has: its PID, current floor, the destination floor it is travelling to, the character it is currently carrying (if any), the word ID of that character, and a done semaphore that the elevator posts when the carrier arrives at its destination.

4. Synchronization Strategy

4.1 Semaphores Used

All synchronization uses POSIX unnamed semaphores (sem_init with pshared=1) embedded in shared memory. The key semaphores are:

- rr_mtx: protects the round-robin word index and word claiming
- adm_mtx: protects all-or-nothing floor capacity reservation
- lc_reg_mtx: protects letter-carrier slot allocation
- comp_mtx: protects the global completion counter (n_done)
- floors[f].task_mtx: protects character-task scanning and claiming on floor f
- floors[f].lc_mtx: protects lc_cnt on floor f
- floors[f].lc_work: counting semaphore signalling new character tasks
- floors[f].sort_work: counting semaphore signalling newly placed characters
- words[i].sort_lock: per-word mutex allowing only one sorter at a time
- elev.mtx: protects all elevator state (queue, load, direction, floor)
- elev.avail: signals the elevator when a new request is added
- lc[i].done: posted by the elevator when carrier i reaches its destination

4.2 Race Condition Prevention

Word claiming is atomic: rr_mtx is held while scanning the list and setting claimed=1. No two word-carriers can claim the same word.

Floor capacity is checked atomically: adm_mtx is held while reading and incrementing word_count on both the arrival floor and the sorting floor simultaneously. If either floor is full, neither counter is modified.

Character claiming is atomic: task_mtx is held while scanning tasks and setting claimed=1. No two letter-carriers can claim the same character task.

Sorting area writes: sort_lock is held by both place_char (letter-carriers placing characters) and do_sort_pass (sorting processes). Only one writer at a time can modify the sorting area of a specific word.

4.3 Critical Sections

The main critical sections and their protecting semaphores are:

- Word selection and claiming → rr_mtx
- Admission capacity check and update → adm_mtx
- Character task scan and claim → floors[f].task_mtx
- Sorting area read/write → words[i].sort_lock
- Elevator queue modification → elev.mtx
- Letter-carrier count update → floors[f].lc_mtx
- Completion counter update → comp_mtx

- Statistics update → stats.mtx

4.4 Deadlock Avoidance

Deadlock is avoided by: (1) never holding more than one mutex at a time when possible; (2) defining a consistent lock acquisition order (adm_mtx → floor locks, never the reverse); (3) using non-blocking sem_trywait in sorting processes so they never block while scanning for available words; (4) using timed semaphore waits (sem_timedwait) instead of indefinite blocking in idle loops.

5. Word Admission Logic

5.1 Round-Robin Selection

A global integer shm->rr stores the next scan position. Word-carrier processes acquire rr_mtx, scan from rr forward (wrapping modulo nw), find the first unclaimed, unadmitted, incomplete word, set its claimed flag, advance rr, and release rr_mtx. This ensures fair, sequential access to the word list across all word-carrier processes.

5.2 Atomic Capacity Checking

After claiming a word, the word-carrier acquires adm_mtx and checks: floors[arrival].word_count + 1 <= max_wf AND floors[sorting].word_count + 1 <= max_wf (using a single check when arrival == sorting). If both checks pass, both counters are incremented atomically within the same critical section. The word's arrival_floor and admitted flag are set. adm_mtx is then released.

5.3 Admission Failure Handling

If admission fails, the word-carrier re-acquires rr_mtx, clears the claimed flag (making the word available for future attempts), releases rr_mtx, increments the retries counter, and sleeps for 10 ms before trying again.

6. Character Transportation and Elevators

6.1 Character Task Generation

When a word is loaded, one CharTask is created per character, storing: word_id, the character itself, its original index (orig_idx), and the destination floor (= sorting floor). Tasks are stored in shm->words[i].tasks[] and initialized with claimed=0 and delivered=0.

6.2 Task Assignment

Letter-carriers on the arrival floor acquire floors[f].task_mtx and scan admitted, incomplete words for unclaimed, undelivered tasks. A random word is selected from candidates, then a random task from that word. The task's claimed flag and carrier_id are set before releasing the mutex. This ensures no two letter-carriers claim the same task.

6.3 Delivery Elevator

The delivery elevator runs the LOOK algorithm: it moves in the current direction (UP or DOWN), dropping off passengers whose destination equals the current floor, then picking up new passengers requesting the current floor (up to capacity). After each floor, it checks whether there are requests or passengers above or below, and reverses direction if needed. When idle, it waits on the avail semaphore until a new request arrives.

A letter-carrier requests the delivery elevator by: (1) acquiring elev.mtx, (2) adding an ElevReq entry (from_floor, to_floor, lc_id) to the sparse request array, (3) releasing elev.mtx, (4) posting elev.avail, and (5) blocking on lc[lc_id].done. The elevator posts lc[lc_id].done when it drops the carrier off at the destination.

6.4 Reposition Elevator

The reposition elevator operates identically to the delivery elevator but uses a separate request queue (shm->repos). Idle letter-carriers (those with no character task) add a request to the reposition elevator with a randomly chosen destination floor (different from the current floor). After arrival, the carrier resumes looking for work on the new floor.

6.5 Idle Letter-Carrier Handling

When a letter-carrier finds no available tasks on its current floor, it: (1) logs the idle state, (2) generates a random destination floor, (3) decrements lc_cnt for the current floor, (4) requests the reposition elevator, (5) blocks on its done semaphore, (6) increments lc_cnt for the new floor, and (7) waits briefly on lc_work (50 ms timeout) before scanning for tasks again. This avoids busy waiting.

7. Sorting Algorithm

7.1 Character Placement

When a letter-carrier delivers a character, it acquires sort_lock for the target word and searches for the first unoccupied, non-fixed slot in area[]. The character is placed there and occ[slot] is set to 1. The delivered flag and delivered_cnt are updated. sort_lock is released, then sort_work is posted to wake sorting processes.

7.2 Sorting Pass (do_sort_pass)

A sorting pass iterates over all slots of a word. For each slot:

- If empty (occ[i] == 0): skip.
- If fixed (fix[i] == 1): skip.
- If area[i] == orig[i]: mark fix[i] = 1 (correct position).
- If area[i] != orig[i]: find the first non-fixed slot j where orig[j] == area[i].
  - If slot j is empty: move character there (clear slot i, set slot j).
  - If slot j is occupied and not fixed: swap area[i] and area[j], then check if either is now fixed.
  - If slot j is fixed: do nothing.

The pass returns 1 (complete) if all fix[] == 1, otherwise 0.

7.3 Fixed Positions

Once fix[i] == 1, the slot is immutable: no character may be placed there, moved from there, or swapped into that position. The letter-carrier's place_char function explicitly skips fixed slots when searching for a placement slot.

7.4 Handling Repeated Characters

For words with repeated characters (e.g., 'apple' with two 'p's), the sorter finds the first non-fixed slot j satisfying orig[j] == area[i] and j != i. If that slot contains the same character value, a swap still results in one position being fixed, gradually converging to the sorted state across multiple passes.

8. Termination Detection

8.1 Completion Flags

Each word has three lifecycle flags: claimed (word-carrier is working on it), admitted (word is active in the system), and completed (all characters sorted). A word is complete when all fix[] == 1, at which point the sorting process sets completed = 1 and increments shm->n_done under comp_mtx.

8.2 Parent Termination Logic

The parent monitor loop checks shm->n_done >= shm->nw every 100 ms. When all words are complete, the loop exits. The parent then sets shm->done = 1, posts the elevator avail semaphores and all floor semaphores to unblock waiting processes, posts all lc[i].done semaphores to release blocked letter-carriers, waits 500 ms, sends SIGTERM to all children, waits an additional 200 ms, and calls waitpid on all child PIDs.

8.3 SIGINT Handling

On Ctrl+C, the SIGINT handler sets g_got_sigint = 1 and shm->done = 1. The monitor loop detects g_got_sigint and exits, triggering the same shutdown sequence described above.
To prevent zombie processes, the parent calls **waitpid** with the **WNOHANG** flag in a loop after sending SIGTERM, collecting the exit status of every child PID. If any child has not exited within 500 ms, the parent sends **SIGKILL** and calls **waitpid** once more to reap the remaining children. This guarantees that no zombie processes are left behind after either normal completion or Ctrl+C.

9. Output File Generation

9.1 Generation Process

After all children have exited, the parent calls write_output(). It sorts word indices by sorting_floor (ascending), breaking ties by word_id (ascending), using a simple $O(n^2)$ sort. It then iterates over the sorted indices, reconstructs each word from area[] (concatenating all occupied slots in order), and writes one line per word in the format: word_id word sorting_floor.

9.2 Sorting Guarantee

The output is sorted first by sorting_floor, then by word_id. This matches the required output format exactly.

9.3 Correctness

When a word is marked completed, all fix[] == 1 and all area[i] == orig[i], so every slot is occupied with the correct character. The concatenation of area[0..len-1] (skipping unoccupied slots, which do not exist at completion) produces the original word.

10. Test Case Scenario

10.1 Input File (input.txt)

The following input file was used for the mandatory test case:

```
101 apple 2
102 process 0
103 kernel 1
104 system 2
105 thread 0
```

10.2 Command Used

```
make
./hw3 -f 3 -w 2 -l 2 -s 2 -c 5 -d 6 -r 6 -i input.txt -o output.txt
```

10.3 Expected Output File (output.txt)

The output file must be sorted by sorting_floor then word_id:

```
102 process 0
105 thread 0
103 kernel 1
101 apple 2
104 system 2
```

10.4 Terminal Output Screenshot

```
fatih@fatih:~/Desktop/230104004090$ make
gcc -Wall -Wextra -g -O0 -D_GNU_SOURCE -c -o main.o main.c
gcc -Wall -Wextra -g -O0 -D_GNU_SOURCE -c -o shared.o shared.c
gcc -Wall -Wextra -g -O0 -D_GNU_SOURCE -c -o word_carrier.o word_carrier.c
gcc -Wall -Wextra -g -O0 -D_GNU_SOURCE -c -o letter_carrier.o letter_carrier.c
gcc -Wall -Wextra -g -O0 -D_GNU_SOURCE -c -o sorting.o sorting.c
gcc -Wall -Wextra -g -O0 -D_GNU_SOURCE -c -o elevator.o elevator.c
gcc -Wall -Wextra -g -O0 -D_GNU_SOURCE -o hw3 main.o shared.o word_carrier.o letter_carrier.o sorting.o elevator.o -lrt
-lpthread
fatih@fatih:~/Desktop/230104004090$ ./hw3 -f 3 -w 2 -l 2 -s 2 -c 5 -d 6 -r 6 -i input.txt -o output.txt
Program is starting...
Input file is being read...
Shared memory is initialized...
Synchronization primitives are created...
Processes are being created...
[PID:149153] Parent process started
--- Initializing Floor 0 ---
[PID:149154] Word-carrier-process_0 initialized on floor 0
[PID:149155] Word-carrier-process_1 initialized on floor 0
[PID:149156] Letter-carrier-process_0 initialized on floor 0
[PID:149157] Letter-carrier-process_1 initialized on floor 0
[PID:149158] Sorting-process_0 initialized on floor 0
[PID:149159] Sorting-process_1 initialized on floor 0
--- Initializing Floor 1 ---
[PID:149160] Word-carrier-process_2 initialized on floor 1
[PID:149161] Word-carrier-process_3 initialized on floor 1
[PID:149162] Letter-carrier-process_2 initialized on floor 1
[PID:149163] Letter-carrier-process_3 initialized on floor 1
[PID:149164] Sorting-process_2 initialized on floor 1
[PID:149165] Sorting-process_3 initialized on floor 1
--- Initializing Floor 2 ---
[PID:149166] Word-carrier-process_4 initialized on floor 2
```

```
--------------------------------------------------
All words have been transported and sorted...
Output file is being created...

System Summary:
Total words: 5
Completed words: 5
Retries: 0
Characters transported: 30
Delivery elevator operations: 36
Reposition elevator operations: 46
Program terminated successfully.
fatih@fatih:~/Desktop/230104004090$
```

10.5 Observed Behavior

Write a brief explanation of what you observed when running the program with the above command. Describe: how words were admitted, how characters were transported by letter-carriers, how the elevator picked up and dropped off carriers, how sorting processes fixed characters, and how the system terminated cleanly.

When the program was executed with the command ./hw3 -f 3 -w 2 -l 2 -s 2 -c 5 -d 6 -r 6 -i input.txt -o output.txt, all five processes types were initialized floor by floor in the correct order, with word-carrier, letter-carrier, and sorting processes spawned per floor before the two elevator processes started. Word-carrier processes immediately began admitting words to their floors using synchronized round-robin selection; all five words were admitted without any retries, confirming that the all-or-nothing admission control functioned correctly. Letter-carrier processes concurrently picked up character tasks from arrival floors and used the delivery elevator to transport them to sorting floors, with the LOOK-algorithm elevator picking up and dropping off multiple carriers per floor visit as seen in the terminal output. Sorting processes on each floor fixed characters incrementally as they arrived, without waiting for all characters of a word to be delivered first, demonstrating correct concurrent sorting. Upon completion of all 30 characters across five words, the system printed the summary (Total words: 5, Completed words: 5, Retries: 0, Characters transported: 30) and generated the correctly sorted output file before terminating cleanly.

11. Challenges and Solutions

11.1 Synchronization of Floor Capacity

Challenge: Ensuring that both the arrival floor and the sorting floor capacity are checked and reserved atomically to prevent two word-carriers from both seeing available capacity and then both admitting words that exceed the limit.

Solution: A single global semaphore (adm_mtx) wraps the entire capacity check-and-increment block for both floors. No other process can modify word_count on any floor while this lock is held.

11.2 Elevator Direction and Spurious 'Arrived' Messages

Challenge: The elevator's prev_floor was initialized to -1, causing it to print 'arrived at floor 0' on the very first iteration before making any movement.

Solution: prev_floor was initialized to elev->cur (which is 0 at startup), preventing the spurious arrival print.

11.3 Sorting Correctness with Repeated Characters

Challenge: Words like 'apple' contain two 'p' characters. A naive swap might cycle indefinitely between the two positions.

Solution: The sorter always picks the first eligible target slot (the first non-fixed slot j where orig[j] == area[i]). After each swap or move, it immediately checks if the new position is correct and fixes it. This guarantees convergence.

11.4 Letter-Carrier Starvation on Empty Floors

Challenge: If all letter-carriers migrate away from a floor that still has pending character tasks, those tasks would never be picked up.

Solution: The parent monitor loop checks lc_cnt for every floor every 100 ms. If any floor reaches lc_cnt == 0 while the system is still running, the parent immediately spawns the configured number (lc_pf) of new letter-carriers on that floor.

References

- POSIX Semaphores: sem_init(3), sem_wait(3), sem_post(3), sem_timedwait(3) – Linux man pages
- POSIX Shared Memory: shm_open(3), mmap(2), munmap(2) – Linux man pages
- Process Management: fork(2), waitpid(2), kill(2), sigaction(2) – Linux man pages
- CSE344 Homework #3 Assignment PDF – Gebze Technical University, 2026
- M. Kerrisk, The Linux Programming Interface: A Linux and UNIX System Programming Handbook, No Starch Press, 2010. – Ch. 48: POSIX Shared Memory; Ch. 53: POSIX Semaphores; Ch. 24: Process Creation (fork); Ch. 26: Monitoring Child Processes (waitpid, SIGCHLD); Ch. 21: Signals